

Quantum Pattern Matching Fast on Average

Ashley Montanaro¹

Received: 19 November 2014 / Accepted: 21 August 2015 / Published online: 27 August 2015
© Springer Science+Business Media New York 2015

Abstract The d -dimensional pattern matching problem is to find an occurrence of a pattern of length $m \times \cdots \times m$ within a text of length $n \times \cdots \times n$, with $n \geq m$. This task models various problems in text and image processing, among other application areas. This work describes a quantum algorithm which solves the pattern matching problem for random patterns and texts in time $\tilde{O}((n/m)^{d/2} 2^{O(d^{3/2} \sqrt{\log m})})$. For large m this is super-polynomially faster than the best possible classical algorithm, which requires time $\tilde{\Omega}(n^{d/2} + (n/m)^d)$. The algorithm is based on the use of a quantum subroutine for finding hidden shifts in d dimensions, which is a variant of algorithms proposed by Kuperberg.

Keywords Quantum algorithms · Quantum query complexity · Pattern matching · Average-case complexity

1 Introduction

One of the most fundamental tasks in computer science is pattern matching: finding some desired data (the *pattern*) within a larger data set (the *text*). This problem has been of interest for decades, both in its own right and as part of more complicated questions in text processing, bioinformatics and image processing.

Here we consider the d -dimensional pattern matching problem, for arbitrary $d = O(1)$. Two examples of this problem are shown in Fig. 1. We are given access to a text T and a pattern P over an alphabet Σ , with $|\Sigma| = q \geq 2$. Our task is to find an instance of P within T , if such an instance exists. That is, writing $[n] := \{0, \dots, n-1\}$ and

✉ Ashley Montanaro
ashley@cs.bris.ac.uk

¹ Department of Computer Science, University of Bristol, Bristol, UK

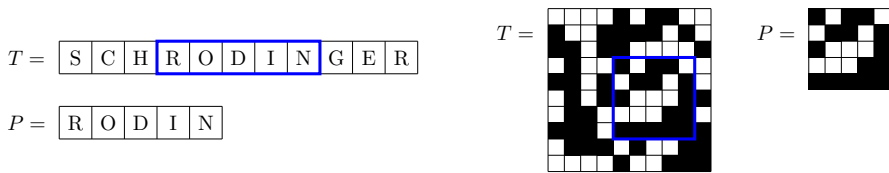


Fig. 1 Examples of 1D and 2D pattern matching problems, with matches highlighted

thinking of T and P as functions $T : [n]^d \rightarrow \Sigma$, $P : [m]^d \rightarrow \Sigma$, we are required to output $s \in [n - m]^d$ such that $T(s + x) = P(x)$ for all $x \in [m]^d$, if such an s exists; otherwise, we should output “not found”. Throughout this work, we call any function of the form $S : [k]^d \rightarrow \Sigma$ a *string*, and think of strings interchangeably as functions or $k \times \cdots \times k$ arrays of elements of Σ . We assume throughout that $m \leq n$.

The classical KMP algorithm of Knuth et al. [20] from 1977 solves the pattern matching problem for $d = 1$ in time $\Theta(n + m)$ in the worst case. This is clearly optimal, as every classical pattern-matching algorithm which is correct on all inputs must inspect every character of the pattern and the text. However, significantly improved runtimes can be achieved for more typical inputs. Consider a model where each character of the text is chosen at random from Σ , and the pattern is either uniformly random too (in which case, if it is long enough, it will not match the text with high probability), or is chosen to be a random substring of the text. A simple algorithm was given by Knuth [20, Section 8] which runs in time $O(n(\log_q m)/m + m)$ with high probability on such random inputs, while still running in time $O(n + m)$ in the worst case. Observe that the average-case runtime is substantially sublinear in n for large m , but never better than $O(\sqrt{n \log n})$.

Shortly after this algorithm was developed, Yao proved an $\Omega((n/m) \log_q m)$ lower bound for the 1-dimensional matching problem, for random text and pattern [34]. The bound extends to give a $\Omega((n/m)^d \log_q m)$ lower bound for the d -dimensional problem [19]. More recently, an algorithm which runs in time $O((n/m)^d \log_q m + m^d)$ for the general d -dimensional problem, for random text and pattern, was given by Kärkkäinen and Ukkonen [19]. This is thus optimal up to the $O(m^d)$ term, which corresponds to preprocessing time for the pattern.

A quantum pattern-matching algorithm for the 1-dimensional case has been presented by Ramesh and Vinay [27], which runs in time $\tilde{O}(\sqrt{n})$ and hence achieves a square-root speedup over the best possible classical algorithm’s worst-case complexity. However, the sublinear classical results mentioned above raise the following question: could there be a quantum pattern-matching algorithm which significantly outperforms its classical counterparts on average-case inputs which are more likely to occur in practice?

1.1 Statement of Results

We give a quantum algorithm which, for most instances of the d -dimensional pattern matching problem, is super-polynomially faster than the best possible classical algorithm.

Theorem 1 Assume $m = \omega(\log n)$. Let $T : [n]^d \rightarrow \Sigma$ be picked uniformly at random. Let $P : [m]^d \rightarrow \Sigma$ be picked either (a) by choosing an arbitrary $m \times \cdots \times m$ substring of T , or (b) by choosing each element of P uniformly at random from Σ . Then there is a quantum algorithm which runs in time $\tilde{O}((n/m)^{d/2} 2^{O(d^{3/2} \sqrt{\log m})})$ and determines which is the case. In case (a), the algorithm also outputs the position at which P matches T . The algorithm fails with probability $O(1/n^d)$, taken over both the choice of T and P , and the algorithm's internal randomness.

Any classical bounded-error algorithm for the same problem must make $\tilde{\Omega}(n^{d/2} + (n/m)^d)$ queries to T and P in total.

The $\tilde{O}$, $\tilde{\Omega}$ notation suppresses factors logarithmic in m and n (see Propositions 8 and 13 below for a more detailed statement of the quantum and classical complexities, respectively). The time complexity is stated in the standard quantum circuit model, assuming that a query to T or P uses time $O(1)$. We can think of T and P as either easily evaluated oracle functions in the query complexity model, or data stored in an efficiently accessible quantum random-access memory [16]. All non-query operations performed by the algorithm contribute only polylogarithmic factors to the time complexity.

Observe that, for any fixed d , $2^{O(d^{3/2} \sqrt{\log m})} = o(m^\epsilon)$ for any $\epsilon > 0$. When m is large, Theorem 1 thus demonstrates a super-polynomial separation between quantum and classical complexity (when m is small, e.g. $O(\log n)$, straightforward use of Grover's algorithm is faster). For example, when $m = \Omega(n)$, we get a quantum algorithm running in time $\tilde{O}(2^{O(d^{3/2} \sqrt{\log n})})$, as opposed to the best classical complexity of $\tilde{\Omega}(n^{d/2})$. The omitted constants in the $O(d^{3/2} \sqrt{\log m})$ term in the exponent are not unreasonably high. For $d = 1$, for example, the algorithm's runtime is $\tilde{O}(\sqrt{n/m} 2^{2.68 \dots \sqrt{\log_2 m}})$. Theorem 1 is a rare example of a super-polynomial average-case separation between quantum and classical computation for a natural problem and a natural distribution on the input. An exponential average-case separation was previously proven [14] for a related problem (an oracular hidden shift problem over $\mathbb{Z}_2^n$, see below), but that problem is arguably less natural than pattern matching.

Theorem 1 is based on a more general pattern matching result, which holds for non-random patterns and texts. In order to state this result more formally, we need some notation. For any string $S : [n]^d \rightarrow \Sigma$, we define a new string $S^{\triangleright k} : [n - k + 1]^d \rightarrow \Sigma^{k^d}$, where $S^{\triangleright k}(s_1, \dots, s_d)$ is equal to the size $k \times \cdots \times k$ substring of S beginning at position $s_1, \dots, s_d$. Formally, for any $f : [n]^d \rightarrow \Sigma$, $k \in \{1, \dots, n\}$, $s \in [n - k + 1]^d$, let $f_{s,k} : [k]^d \rightarrow \Sigma$ be defined by $f_{s,k}(z_1, \dots, z_d) = f(s_1 + z_1, \dots, s_n + z_n)$. Then

$$S^{\triangleright k}(s_1, \dots, s_d) = S_{s,k}.$$

An example of this operation is shown in Fig. 2. Note that we always consider $S^{\triangleright k}$ to be a string over the alphabet Σ^{k^d} ; equivalently, a function $S^{\triangleright k} : [n - k + 1]^d \rightarrow \Sigma^{k^d}$. Let the *injectivity length* of S , $\nu(S)$, be the minimal k such that $S^{\triangleright k}$ is injective (i.e. all of its values are distinct).

We now consider the string $(S^{\triangleright k})_{s,m} : [m]^d \rightarrow \Sigma^{k^d}$ for $k \in \{1, \dots, n\}$, $m \in \{1, \dots, n - k - 1\}$, $s \in [n - k - m + 2]^d$. Define the *m-injectivity length* of S , $\nu(S, m)$,

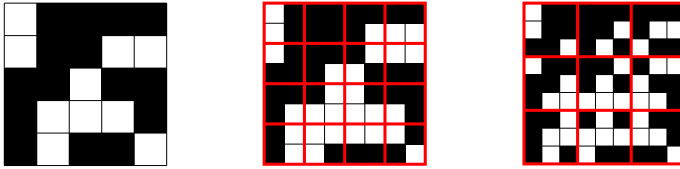


Fig. 2 Converting a non-injective 2D string S (LHS) into an injective string S^3 (RHS). The centre string S^2 is not injective, so $v(S) = 3$. In addition, $v(S, 2) = 3$ as the 2×2 substring of S^2 in the middle of the top row is not injective

to be the minimal k such that $(S^k)_{s,m}$ is injective for all s . Thus $v(S, m) \leq k$ if every $m \times \dots \times m$ substring of S^k is injective. Observe that, for any m , $v(S, m) \leq v(S) \leq n$, but $v(S, m)$ can sometimes be much smaller than $v(S)$. For example, if $S : [n]^d \rightarrow \Sigma$ is constant, $v(S, 1) = 1$, but $v(S) = n$.

Then the most general result we have is as follows:

Theorem 2 Fix $d = O(1)$. Let $T : [n]^d \rightarrow \Sigma$ and $P : [m]^d \rightarrow \Sigma$ satisfy $v(T, m), v(P) \leq v \leq m/2$, for some v . Further assume that, for every offset s such that P does not match T at that offset, the fraction of positions $x \in [m]^d$ where $P(x) \neq T(x + s)$ is at least γ . Then there is a bounded-error quantum algorithm which outputs $s \in [m]^d$ such that P matches T at offset s , if such an s exists; otherwise, the algorithm outputs “not found”. The algorithm makes

$$O\left(\left(\frac{n \log^2 m 2^{\sqrt{(2 \log_2 3)d \log_2 m}}}{m}\right)^{d/2} \left(v^d \log m 2^{\sqrt{(2 \log_2 3)d \log_2 m}} + \frac{1}{\sqrt{\gamma}}\right)\right)$$

queries to each of T and P . The runtime is the same up to a polylog(m) factor.

Theorem 2 may appear somewhat hard to digest. The intuition is that the algorithm is efficient, i.e. has runtime close to $O((n/m)^{d/2})$, when: the strings formed by concatenating all short substrings of both T and P are injective; and offsets where there is no match can be efficiently tested and discarded. The algorithm can thus be seen as achieving a speedup in a scenario somewhat similar to that considered in the field of property testing [23], where it has the promise that each potential match is either actually a match, or is far from being a match.

The complexity parameters of Theorem 2 are essentially optimal for, say, $v = 1$, up to terms of the form $2^{O(\sqrt{\log m})}$, which is $o(m^\epsilon)$ for any $\epsilon > 0$. Indeed, it is easy to show the following bound using standard techniques: any bounded-error quantum algorithm which solves the problem described in Theorem 2 must make $\Omega((n/m)^{d/2}/\sqrt{\gamma})$ queries in the case where P is injective and completely known in advance, and $v(T, m) = 1$. By comparison, any randomised classical pattern matching algorithm for the same problem must make $\Omega((n/m)^d/\gamma)$ queries. For the proof of these bounds, see Lemma 10 below.

1.2 Techniques

Theorem 2 is ultimately based around the use of a quantum algorithm for finding hidden shifts in injective functions $f : \mathbb{Z}_{2^n}^d \rightarrow \Sigma$. The algorithm is a variant of algorithms of Kuperberg [21]. Kuperberg’s work described several algorithms: two for finding hidden shifts in injective functions $f : \mathbb{Z}_{2^n} \rightarrow \Sigma$, and one for finding hidden shifts in general abelian groups. The algorithm given here achieves essentially the same asymptotic complexity as the best algorithm given in [21], and appears somewhat simpler to analyse. In particular, we include a full proof of its correctness and complexity.

To use the algorithm, we first make the pattern and text injective. This is similar to the “injectivisation” idea used by Gharibi [15] in the context of quantum algorithms for abelian hidden shift problems, but here we need a slightly different notion, as used by Knuth [20], to ensure we preserve matching after injectivisation. We then apply the hidden shift algorithm by guessing an offset where the pattern matches the text. If our guess is fairly close, then the algorithm succeeds in finding the actual offset where the pattern matches. This guessing process is then wrapped within the use of the bounded-error variant of Grover’s search algorithm [18] to obtain the final result. Theorem 1 is then derived by simply calculating the quantities ν , γ that occur in Theorem 2 for random strings.

Kuperberg showed in [21] that, based on a similar idea of guessing offsets, his algorithms gave a super-polynomial quantum speedup for the task of finding an injective pattern of length m , promised to be hidden in an injective text of length $2m$. The contribution here is thus to generalise this idea to arbitrary dimensions $d > 1$, to remove the restriction on the length of the text, and to relax the injectivity constraint. We also modify the promise that the pattern is guaranteed to be contained in the text to the promise that any non-matches can be tested efficiently. Observe that a constraint of this form is required if one seeks a runtime which is $o(m^{d/2})$. Imagine we are told an offset at which the pattern is claimed to match the text. If we have no lower bound on the number of positions at which it does not match the text if the claim is false, then verifying this claimed match would require $\Omega(m^{d/2})$ quantum queries in the worst case [4].

1.3 Prior Work

Pattern matching is a fundamental algorithmic task, and has been studied in a number of different contexts.

1.3.1 Pattern Matching

As well as the KMP algorithm already mentioned, another approach frequently used in practice classically is the Boyer-Moore algorithm [5], which achieves good performance for random inputs when the alphabet size is large. More recent classical work has pursued a number of other directions, such as approximate matching and search in compressed strings. For surveys of the (now vast) classical pattern matching literature, see for example [10, 25].

Pattern matching has also been considered in the context of quantum computation. Grover’s algorithm [17] can be seen as searching for a pattern of length 1 in a text of length n using $O(\sqrt{n})$ queries to the text. The algorithm can be used naïvely to find a pattern of length m with a complexity of $O(\sqrt{nm})$ queries. However, Ramesh and Vinay [27] gave an improved algorithm which uses $O(\sqrt{n})$ queries in the worst case. Their algorithm is based around the use of the powerful classical concept of deterministic sampling [33]. It is easy to see that this complexity is optimal in the worst case up to logarithmic factors, using standard lower bounds on the quantum query complexity of unstructured search [4]. The algorithm of Ramesh and Vinay achieves a faster runtime than the algorithms described here in the case that m is small (e.g. $O(\log n)$).

Curtis and Meyer describe an approach towards efficient quantum search for patterns (“templates”) within 2D images [11]. This approach is based around the use of the quantum Fourier transform to compute correlations between the template and the image. As noted in [11], the algorithm is not complete and many details remain to be worked out before an efficient algorithm could be obtained.

1.3.2 Hidden Shifts

There is now a fairly substantial body of work in the quantum setting on the closely related problem of finding hidden shifts. In an abstract setting, one is given access to two injective functions $f, g : G \rightarrow X$, for some abelian group G and some set X , with the promise that $g(x) = f(x + s)$ for some $s \in G$, where $+$ is addition in the group G . The goal is to find s . It is known that this problem can be solved with only $O(\log |G|)$ quantum queries to G [12]. However, for certain groups G it remains unknown whether there is a quantum algorithm which is similarly efficient with respect to time (i.e. runs in time $O(\text{polylog } |G|)$), and this is considered to be a major open problem.

The case $G = \mathbb{Z}_n$ is the most relevant to our work here, which is equivalent to the hidden subgroup problem for the dihedral group [31]. Algorithms to solve this problem have been given by Kuperberg [21, 22] (whose work we will use and adapt below) and Regev [28]. These algorithms are all super-polynomially faster than the best possible classical algorithm for this problem: Kuperberg’s algorithms run in time $2^{O(\sqrt{\log n})}$, while Regev’s is almost as fast, running in time $2^{O(\sqrt{\log n \log \log n})}$. However, Kuperberg’s algorithms use space $2^{O(\sqrt{\log n})}$, while Regev’s algorithm uses space only $\text{poly}(\log n)$. The more recent algorithm of [22] uses only $O(\log n)$ quantum space, but $2^{O(\sqrt{\log n})}$ classical space. Our focus here is on optimising time complexity, so we base our algorithm on Kuperberg’s.

The hidden shift problem has been studied for other groups G too. Friedl et al. [13] have given an efficient quantum algorithm running in time $\text{poly}(\log |G|)$ for the case of $G = \mathbb{Z}_p^n$, where p is a fixed prime and n grows. A different generalisation, which can be seen as interpolating between the abelian hidden subgroup problem and the dihedral hidden subgroup problem, was studied by Childs and van Dam [8].

A number of works have studied a slightly different scenario in which one relaxes the injectivity constraint, but replaces it with complete knowledge of f . That is, one

is given oracle access to a function $g : G \rightarrow X$, such that $g(x) = f(x + s)$ for some known function f , and is required to determine s . The complexity of the problem then depends on f . This problem was studied for $G = \mathbb{Z}_n$ for certain functions f by van Dam et al. [32], as well as by Moore et al. [24] for prime n . More recently, other works have considered the case $G = \mathbb{Z}_2^n$ in detail [9, 14, 26, 29, 30], characterising the complexity of the problem for many families of functions f .

To convert non-injective strings into injective strings, we concatenate adjacent symbols within the string. A similar idea was recently used by Gharibi to convert non-injective hidden shift problems over an arbitrary group into injective hidden shift problems [15]. The general framework takes a function $f : G \rightarrow X$ and a k -tuple $V \in G^k$, and defines a new function $f_V(x) = (f(x \cdot v_1), \dots, f(x \cdot v_k))$, where $\cdot$ is the group multiplication operation. In the hidden shift problem, if g is equal to f up to a shift (i.e. multiplication by an unknown group element s), then g_V is equal to f_V up to the same shift s . Gharibi showed that, for any choice of V with $k = \Omega(\log |G|)$, if f is picked at random then the probability that f_V is not injective is low. This was then used to give an alternative proof that the quantum query complexity of the hidden shift problem over $\mathbb{Z}_2^n$ is low for most functions. The injectivisation procedure over the group $\mathbb{Z}_n$ used here is slightly different: in order to preserve the property of the pattern matching the text, we do not allow the shifts to wrap around and only consider the set $V = \{1, \dots, k\}$.

The hidden shift problem over $\mathbb{Z}_n$ has also been studied classically. In particular, Andoni et al. [2] have considered a noisy variant where one has access to two boolean functions $f, g : \mathbb{Z}_n \rightarrow \{0, 1\}$ such that $g(x) = f(x + s) + r$ for some shift s , and some string $r \in \{0, 1\}^n$ of random bits where the probability that each bit of r is equal to 1 is independent and equal to η , for some fixed constant η . The goal is again to find s . A practical motivation for this problem comes from GPS synchronisation. In the case where the values taken by f are uniformly random, it is shown in [2] that the problem can be solved in sublinear time; their algorithm runs in time $O(n^{0.641})$. This algorithm is not that far from optimal, as the hidden shift problem over $\mathbb{Z}_n$ has a lower bound of $\Omega(\sqrt{n})$ queries [3].

1.4 Organisation

We begin, in Sect. 2, by describing how a quantum algorithm for the hidden shift problem can be used to obtain a general pattern-matching algorithm (Theorem 2). Section 3 contains the calculations showing that this can be applied to random strings (Theorem 1). In Sect. 4, we prove the required classical lower bounds to complete the proof of Theorem 1, and also prove quantum lower bounds showing that our algorithms are not too far from optimal. For completeness, in this section we also give a classical algorithm matching the classical lower bound. Section 5 describes the quantum algorithm for the hidden shift problem. We conclude in Sect. 6.

2 Quantum Pattern Matching Based on Finding Hidden Shifts

We will need the following simple lemma, whose proof follows immediately from amplitude amplification [6].

Lemma 3 Assume we have query access to $S, T : [m] \rightarrow \Sigma$ such that either $S = T$, or $|\{x \mid S(x) \neq T(x)\}| \geq \gamma m$, for some $0 < \gamma \leq 1$. Then there is a quantum algorithm **Check** such that: in the first case, **Check** accepts with certainty; in the second case, **Check** rejects with probability at least $2/3$; **Check** makes $O(1/\sqrt{\gamma})$ queries. The runtime is the same up to a $\text{polylog}(m)$ factor.

The technical core of our algorithm is the following result:

Theorem 4 Let $d = O(1)$ and let X be an arbitrary finite set. Let $f : \mathbb{Z}_{2^n}^d \rightarrow X$ and $g : \mathbb{Z}_{2^n}^d \rightarrow X$ be injective functions such that $\Pr_x[g(x) \neq f(x + s)] = O(n^{-2}2^{-\sqrt{(2 \log_2 3)dn}})$ for some $s \in \mathbb{Z}_{2^n}^d$. Then there is a quantum algorithm which outputs s with bounded error using $O(n2^{\sqrt{(2 \log_2 3)dn}}) = O(n2^{1.781 \dots \sqrt{dn}})$ queries to each of these functions. The runtime is the same up to a $\text{poly}(n)$ factor.

We prove Theorem 4 later, in Sect. 5. We show here that it implies Theorem 2, which we restate for convenience.

Theorem 2 (restated) Fix $d = O(1)$. Let $T : [n]^d \rightarrow \Sigma$ and $P : [m]^d \rightarrow \Sigma$ satisfy $v(T, m), v(P) \leq v \leq m/2$, for some v . Further assume that, for every offset s such that P does not match T at that offset, the fraction of positions $x \in [m]^d$ where $P(x) \neq T(x + s)$ is at least γ . Then there is a bounded-error quantum algorithm which outputs $s \in [m]^d$ such that P matches T at offset s , if such an s exists; otherwise, the algorithm outputs “not found”. The algorithm makes

$$O\left(\left(\frac{n \log^2 m 2^{\sqrt{(2 \log_2 3)d \log_2 m}}}{m}\right)^{d/2} \left(v^d \log m 2^{\sqrt{(2 \log_2 3)d \log_2 m}} + \frac{1}{\sqrt{\gamma}}\right)\right)$$

queries to each of T and P . The runtime is the same up to a $\text{polylog}(m)$ factor.

Proof Let m' be the largest power of 2 less than or equal to $m - v$. The algorithm is based on the following procedure **RoughCheck**, which takes as input a shift $t \in [n - v - m' + 2]^d$ and a tolerance $\epsilon \in (0, 1]$, and is designed to accept if t is quite close to a position t' at which P matches T :

1. Apply the algorithm of Theorem 4 to $T' := (T^{\triangleright v})_{t, m'}$ and $P' := (P^{\triangleright v})_{0, m'}$. Let $\ell \in [m']^d$ be the offset output by the algorithm. If there exists i such that $\ell_i > \epsilon m'$, reject.
2. Otherwise, apply **Check** to $T_{t+\ell, m}$ and P , and accept if and only if it accepts.

In this definition, the notation is as used in Sect. 1.1: thus $(T^{\triangleright v})_{t, m'}$ denotes the $m' \times \dots \times m'$ substring of $T^{\triangleright v}$ starting at offset t , and $(P^{\triangleright v})_{0, m'}$ is the first $m' \times \dots \times m'$ characters of $P^{\triangleright v}$. Note that T' and P' are injective, so Theorem 4 can indeed be applied to them. It is immediate that **RoughCheck** uses $O(v^d \log m 2^{\sqrt{(2 \log_2 3)d \log_2 m}} + 1/\sqrt{\gamma})$ queries, where the $O(v^d)$ term comes from simulating a query to $T^{\triangleright v}$ using v^d queries to T . We now show that, for $\epsilon = O((\log_2^{-2} m')2^{-\sqrt{(2 \log_2 3)d \log_2 m'}})$, **RoughCheck** is a bounded-error verifier for the property of P matching T at some offset $t' \in [n - m + 1]^d$, where $t_i \leq t'_i \leq t_i + \epsilon m'$ for all $i \in \{1, \dots, d\}$. Call this property ϵ -matching.

First assume there does exist such an offset t' . Then $P^{\triangleright v}$ also matches $T^{\triangleright v}$ at the same offset. Taking addition modulo m' in each dimension,

$$|\{x \in [m']^d : P'(x) \neq T'(x + t' - t)\}| \leq \sum_{i=1}^d (t'_i - t_i)(m')^{d-1} = O((m')^d \epsilon),$$

where the first inequality is a rough bound on the size of the complement of one d -dimensional cube within another, in terms of $(d-1)$ -dimensional slices. The algorithm of Theorem 4 therefore outputs $\ell = t' - t$ with bounded failure probability, and if it does so then **Check** accepts with certainty given the two strings $T_{\ell, m}$ and P . On the other hand, if there is no such offset t' , there are two possibilities: the algorithm of Theorem 4 could output a correct match between P and T , but at an offset t' which fails to satisfy $t_i \leq t'_i \leq t_i + \epsilon m'$ for at least one i ; or otherwise, the algorithm could output an incorrect claimed match. In the former case, this will be detected by the check in step 1. In the latter case, **Check** will detect this with bounded error. We therefore see that, if P ϵ -matches T at offset t , **RoughCheck** accepts except with bounded failure probability; whereas if P does not ϵ -match T at offset t , **RoughCheck** rejects except with bounded failure probability.

If we pick t at random from $[n - v - m' + 2]^d$, and P matches T at at least one offset, the probability that P matches T at some offset t' , where $t_i \leq t'_i \leq t_i + \epsilon m'$, is at least $(\epsilon m'/n)^d$. Applying the bounded-error version of Grover's search algorithm [18], we can find a position at which P ϵ -matches T with $O((n/(\epsilon m'))^{d/2})$ uses of **RoughCheck**, and again with bounded failure probability. Once such a position is found, one more use of the algorithm of Theorem 4 suffices to output the offset within this range at which P matches T . The claimed result for the number of queries follows, substituting the value of ϵ back in and using $m' = \Omega(m)$. The argument for the runtime bound is similar. $\square$

Note that there is also an algorithm which does not need to know the value of v in advance, at the expense of a small additional runtime factor. We simply run the algorithm of Theorem 2 multiple times, doubling a guess for v each time. Each time that the algorithm claims we have a match, we can use **Check** to determine if it really is a match. To achieve a sufficiently small probability of failure, we need to repeat **Check** at most $O(\log v)$ times. We can also get an algorithm with no dependence on v if we make some slightly different assumptions (cf. Lemma 10 below for why these assumptions are necessary).

Theorem 5 *Let $T : [n]^d \rightarrow \Sigma$ and $P \in [m]^d \rightarrow \Sigma$ satisfy $v(T), v(P) \leq v \leq m/2$, for some v . Further assume that P matches T at some position i (necessarily unique). Then there is a bounded-error quantum algorithm which outputs i and makes*

$$O\left(\left(\frac{n}{m}\right)^{d/2} v^d \log^{d+1} m 2^{\sqrt{(2 \log_2 3)d \log_2 m}(d/2+1)}\right)$$

queries to each of T and P . The runtime is the same up to a $\text{polylog}(m)$ factor.

Proof Let m' be the largest power of 2 less than or equal to $m - v$. The algorithm is based on the following procedure **RoughCheck2**, which takes as input a shift $t \in [n - v - m' + 2]^d$ and a tolerance $\epsilon \in (0, 1]$:

1. Apply the algorithm of Theorem 4 to $T' := (T^{\triangleright v})_{t, m'}$ and $P' := (P^{\triangleright v})_{0, m'}$. Let ℓ be the offset output by the algorithm. If there exists i such that $\ell_i > \epsilon m'$, reject.
2. If $T^{\triangleright v}(t + \ell) = P^{\triangleright v}(0)$, output ℓ . Otherwise, reject.

The analysis is the same as for Theorem 2, replacing **RoughCheck** with **RoughCheck2**, which uses $O(v^d \log m 2^{\sqrt{(2 \log_2 3)d \log_2 m}})$ queries (and has no dependence on γ). The key difference is that, if $T^{\triangleright v}(t + \ell) \neq P^{\triangleright v}(0)$, we can be sure that $t + \ell$ is not the position where P matches T . This follows from injectivity of $T^{\triangleright v}$ and $P^{\triangleright v}$ and the fact that P is indeed contained somewhere within T . $\square$

3 Pattern Matching in Random Strings

We now show that Theorem 2 can be applied to random patterns and texts. This simply involves calculating the parameters required to apply the theorem. First we show that random strings can be made injective by only considering a small number of subsequent positions. A similar result was previously shown in a more general context by Gharibi [15], using a different notion of injectivisation. Recall that we write $q = |\Sigma|$.

Lemma 6 *Let $S : [n]^d \rightarrow \Sigma$ be uniformly random. Then $\Pr[v(S) \geq (3d \log_q n)^{1/d}] \leq 1/n^d$.*

Proof Consider the string $S^{\triangleright k}$ for arbitrary k . For any offset $s \in [n - k]^d$ and non-zero $\delta \in [n]^d$, the probability over S that $S^{\triangleright k}(s) = S^{\triangleright k}(s + \delta)$ is

$$\Pr_S \left[\bigwedge_{t \in s + [k]^d} (S(t) = S(t + \delta)) \right].$$

This probability is exactly q^{-k^d} , whether or not there exist a pair $t, t + \delta$ that are both contained in $s + [k]^d$. To see why, observe that we can think of choosing S by fixing its values $S(x)$ one by one, in some arbitrary order such that if $x_i \leq y_i$ for all $i \in \{1, \dots, d\}$, $S(x)$ is fixed before $S(y)$. Then, however the value $S(t)$ was chosen, the value of $S(t + \delta)$ is uniformly random. So $\Pr_S[S(t) = S(t + \delta)] = 1/q$. Taking the union bound over all possible choices of s and δ , we obtain an upper bound of $n^{2d} q^{-k^d}$ on the probability that $S^{\triangleright k}$ is non-injective. Choosing $k = \lceil (3d \log_q n)^{1/d} \rceil$ gives the claimed bound. $\square$

We can also show that random strings are unlikely to be close to matching at any offset.

Lemma 7 *Let $T : [n]^d \rightarrow \Sigma$ be arbitrary, and let $P : [m]^d \rightarrow \Sigma$ be uniformly random. Then the probability that there exists an offset $s \in [n - m]^d$ such that $|\{t : P(t) = T(s + t)\}| \geq (3/4)m^d$ is at most $n^d e^{-m^d/8}$.*

Proof For a fixed offset s , using a Chernoff bound we have

$$\Pr_P[|\{t : P(t) = T(s+t)\}| \geq m^d/q + \delta] \leq e^{-2\delta^2/m^d}.$$

Taking a union bound over all offsets s ,

$$\Pr_P[\exists s, |\{t : P(t) = T(s+t)\}| \geq m^d/q + \delta] \leq n^d e^{-2\delta^2/m^d}.$$

Using $q \geq 2$ and taking $\delta = m^d/4$,

$$\Pr_P[\exists s, |\{t : P(t) = T(s+t)\}| \geq (3/4)m^d] \leq n^d e^{-m^d/8}.$$

□

If m satisfies $m^d \geq 16d \ln n$, the bound obtained from Lemma 7 is at most $1/n^d$. Note that a meaningful bound cannot be found for m significantly smaller than this. If we take a random text $T : [n]^d \rightarrow \Sigma$, and divide it up into $(n/m)^d$ blocks of size m^d , the probability that an arbitrary pattern fails to match a given block is $1 - q^{-m^d}$, so the probability that it fails to match all blocks is

$$(1 - q^{-m^d})^{(n/m)^d} \leq e^{-n^d/(m^d q^{m^d})}.$$

This probability is small for $m^d = O(\log n)$. That is, if the pattern is too short, it is likely to “unintentionally” match the text somewhere.

Combining Lemmas 6 and 7, and inserting these parameters into Theorem 2, we get the following result.

Proposition 8 *Let $m = \omega(\log n)$ and fix $d = O(1)$. Let $T : [n]^d \rightarrow \Sigma$ be picked uniformly at random. Let $P : [m]^d \rightarrow \Sigma$ be picked either (a) by choosing an arbitrary $m \times \cdots \times m$ substring of T , or (b) by choosing each element of P uniformly at random from Σ . Then there is a quantum algorithm which makes*

$$O\left(\left(\frac{n}{m}\right)^{d/2} 2^{(d/2+1)\sqrt{(2\log_2 3)d \log_2 m}} \log^{d+1} m \log n\right)$$

queries to T and P and determines which is the case. The runtime is the same up to a polylog(m) factor. The algorithm fails with probability $O(1/n^d)$ over the choice of T and P , and with an arbitrarily small probability over its own internal randomness. In case (a), the algorithm also outputs the position at which P matches T .

This is the first part of Theorem 1.

4 Lower Bounds and Classical Upper Bounds

We now prove nearly matching quantum and classical lower bounds. We will actually lower-bound the complexity of the following variant of the pattern matching problem. We are given access to a pattern $P : [m]^d \rightarrow \Sigma$, and a text $T : [n]^d \rightarrow \Sigma$. We are promised that either there is a unique offset at which P matches T , or there is no such offset. Our task is to determine which is the case. Call this the *pattern detection* problem. As with the quantum upper bounds discussed earlier, we impose the additional promise that, for every offset s such that P does not match T at that offset, the fraction of positions $x \in [m]^d$ where $P(x) \neq T(x + s)$ is at least γ .

To prove a quantum lower bound for this problem, we will use the following result of Ambainis [1].

Theorem 9 (Ambainis [1]) *For any $f : S \subseteq \{0, 1\}^n \rightarrow \{0, 1\}$, let $X, Y \subseteq S$ be two sets of inputs such that $f(x) \neq f(y)$ for all $x \in X$ and $y \in Y$. Further let $R \subseteq X \times Y$ be such that*

1. *For every $x \in X$, there exist at least μ different $y \in Y$ such that $(x, y) \in R$.*
2. *For every $y \in Y$, there exist at least μ' different $x \in X$ such that $(x, y) \in R$.*
3. *For every $x \in X$ and $i \in \{1, \dots, n\}$, there are at most λ different $y \in Y$ such that $(x, y) \in R$ and $x_i \neq y_i$.*
4. *For every $y \in Y$ and $i \in \{1, \dots, n\}$, there are at most λ' different $x \in X$ such that $(x, y) \in R$ and $x_i \neq y_i$.*

Then any quantum algorithm computing $f(x)$ with bounded failure probability for all $x \in S$ uses $\Omega\left(\sqrt{\frac{\mu\mu'}{\lambda\lambda'}}\right)$ queries to the bits of x .

First we show that the quantum pattern matching algorithm given here is not far from optimal, and give a corresponding classical lower bound.

Lemma 10 *Any bounded-error quantum algorithm for the pattern detection problem must make $\Omega((n/m)^{d/2}/\sqrt{\gamma})$ queries, even if P is injective, $v(T, m) = 1$, and P is completely known in advance. Any randomised classical pattern matching algorithm for the same problem must make $\Omega((n/m)^d/\gamma)$ queries.*

Proof Both the quantum and classical lower bounds are based on the same hard input distribution. We use the alphabet $\Sigma = [2m]^d$. Set $P(x_1, \dots, x_d) = (x_1, \dots, x_d)$ and fix $n = mp$ for some integer p . Divide T into p^d blocks of size $m \times \dots \times m$, with each block initially being equal to P . Within each block, either change an arbitrary γ fraction of the elements of T by adding m to each component of the sequence, or change none of them. Then P only matches T at one or more position at offsets given by blocks, and within each block except one fails to match T at a γ fraction of positions. The only information given by querying elements of the text is whether that element is equal to the corresponding element of P , or not. We can therefore think of the text as an n^d -bit string, which is divided into p^d blocks of size $m \times \dots \times m$; within each block the string either takes the value 1 at a γ fraction of positions, or at no positions. The goal is to determine whether there exists a block where the string

is equal to 0 at all positions. This is equivalent to evaluating a 2-level OR-AND tree with a promise on the number of 1's in each block.

For the quantum lower bound, we now apply Theorem 9. Let X be the set of all bit-strings with exactly one block containing only 0's, and all other blocks containing γm^d 1's; and let Y be the set of all bit-strings with all blocks containing γm^d 1's. Finally let R be the set of all pairs $(x, y) \in X \times Y$ such that x and y only differ within exactly one block. Then one can readily calculate, in the notation of Theorem 9, that $\mu = \binom{m^d}{\gamma m^d}$, $\mu' = p^d$, $\lambda = \binom{m^d-1}{\gamma m^d-1}$, $\lambda' = 1$. We therefore obtain a lower bound of $\Omega(\sqrt{\mu\mu'}/(\lambda\lambda')) = \Omega(\sqrt{p^d/\gamma}) = \Omega((n/m)^{d/2}/\sqrt{\gamma})$ queries.

For the classical lower bound, we use the Yao principle that it suffices to prove a lower bound on deterministic algorithms which succeed on most inputs picked from some probability distribution (here, the distribution described above). Imagine that the output should be 0. The algorithm cannot be confident that this is the case until it has seen a 1 in each of the p^d blocks. But, within each block, the number of queries required to do so is $\Omega(1/\gamma)$. Multiplying these bounds gives the claimed result. $\square$

If we have no a priori limitation on γ , it can be as low as $1/m^d$, so in this case we have a bound of $\Omega(\sqrt{n})$ for $d = 1$. This is achieved, up to polylogarithmic factors, by the algorithm of Ramesh and Vinay [27].

We now give a general classical lower bound for the pattern detection problem for the case of injective functions.

Lemma 11 *Any bounded-error classical algorithm which solves the pattern detection problem for all injective functions $P : [m]^d \rightarrow \Sigma$ and $T : [n]^d \rightarrow \Sigma$ must make at least $\Omega(n^{d/2} + (n/m)^d + 1/\gamma)$ queries in total.*

Part of the bound achieved by Lemma 11 is similar to a result of Batu et al. [3], who proved an $\Omega(\sqrt{n})$ lower bound for the hidden shift problem where $f, g : \mathbb{Z}_n \rightarrow \mathbb{Z}$ and we are promised that $g(x) = f(x + s)$ for some $s \in \mathbb{Z}_n$.

Proof We first observe that lower bounds of $\Omega((n/m)^d)$ and $\Omega(1/\gamma)$ are easy. For the former, we divide the text into $(n/m)^d$ contiguous $m \times \dots \times m$ blocks, and impose the promise that the pattern matches the text within one of the blocks. Then the number of queries used by any classical algorithm which identifies which block this is must be lower bounded by a quantity proportional to the number of blocks. For the latter, if we promise that the pattern either matches the text at some known offset, or has a γ fraction of its entries not matching, to determine which is the case requires $\Omega(1/\gamma)$ queries to the text.

So it remains to prove an $\Omega(n^{d/2})$ lower bound. For the proof we use the alphabet $\Sigma = [n^d]$. Consider two distributions $\mathcal{D}_0, \mathcal{D}_1$. In the first distribution, T is a uniformly random permutation of $[n^d]$, and P is formed by choosing m integers from $[n^d]$ at random and then randomly permuting them. In the second distribution, T is formed in the same way, and P is formed by taking a random sub-block within T of size $m \times \dots \times m$. We show that any deterministic classical algorithm making $o(n^{d/2})$ queries cannot distinguish between $\mathcal{D}_0$ and $\mathcal{D}_1$. By the Yao principle, this suffices to prove the corresponding bound for randomised algorithms.

Imagine the algorithm makes K queries to T and L queries to P . Define a collision to be the result of some query to T which equals the result of some previous query to P , or vice versa. Then if the classical algorithm has not seen any collisions when it terminates, the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$, conditioned on the query results, are indistinguishable. It therefore suffices to upper-bound the probability that the algorithm finds a collision. Each result of a query to P or T that is not a collision gives no additional information about P or T . Therefore, until at least one collision is found, we can assume that the choice of subsequent queries does not depend on previous queries. In particular, we can assume that all the queries to P are made first. Then, using a union bound, the probability that one of the queries to T is a collision is at most

$$\frac{L}{n^d} + \frac{L}{n^d - 1} + \cdots + \frac{L}{n^d - K + 1}.$$

Assuming that $K \leq n^d/2$, this is at most $2KL/n^d$. Therefore, to see a collision with probability at least $1/2$, we need $KL = \Omega(n^d)$, implying $K + L = \Omega(n^{d/2})$. $\square$

For completeness, we describe a classical algorithm which matches the bound of Lemma 11.

Theorem 12 *Let $P : [m]^d \rightarrow \Sigma$ and $T : [n]^d \rightarrow \Sigma$ be injective. Then there is a bounded-error classical algorithm which finds P within T , if it exists, using $O(n^{d/2} + (n/m)^d + 1/\gamma)$ queries.*

Proof The algorithm proceeds as follows. Read in the k^d elements of the $k \times \cdots \times k$ substring $P_{0,k}$, for some k to be determined. Then divide the text into $O((n/k)^d)$ contiguous blocks of size at most $k \times \cdots \times k$ and read the entry of the text at position $(0, \dots, 0)$ within each block. If P is contained within T , exactly one of these characters will match one of the characters previously read from P . Once a matching character has been found, the algorithm samples $O(1/\gamma)$ elements from the pattern and the text in the neighbourhood of that character to be convinced that the pattern does indeed match at that offset. To minimise the overall bound, set $k = \min\{\sqrt{n}, m\}$. $\square$

4.1 A Classical Lower Bound for Random Strings

We finally complete the proof of Theorem 1 by giving the promised lower bound on the classical query complexity of pattern matching in random strings.

Proposition 13 *Fix $d = O(1)$. Any bounded-error classical algorithm which solves the pattern detection problem for random functions $P : [m]^d \rightarrow \Sigma$ and $T : [n]^d \rightarrow \Sigma$ must make at least $\Omega((n/m)^d \log_q m + n^{d/2}/\sqrt{\log_q n})$ queries in total, where $q = |\Sigma|$.*

Proof An $\Omega((n \log_q m)/m)$ lower bound on the complexity of this problem was shown by Yao [34] for the case $d = 1$, which can be generalised to arbitrary d [19]. To prove the second part of the bound, we use Lemma 11. Assume for simplicity that n and m are each integer multiples of $b := (3d \log_q n)^{1/d}$. Take an instance of the pattern detection problem and impose the constraint that the pattern is promised to

match at an offset which is an integer multiple of b in each dimension (this can only make the problem easier). Divide both the pattern and the text into blocks of $b \times \dots \times b$ characters, and concatenate the characters within each block to make a “megacharacter” of a larger alphabet Σ' with $|\Sigma'| = q^{b^d} = n^{3d}$. If P and T are uniformly random, then each of the megacharacters will also be picked uniformly at random from Σ' . On the other hand, if P matches T at some offset, then each of the megacharacters will be random, except for those where P matches T . Aside from these positions, all of the other characters in T and P will be unique except with probability at most $\binom{(n/b)^d + (m/b)^d}{2} n^{-3d} = O((nb^2)^{-d}) = O(n^{-d})$. Therefore, the $\Omega(n^{d/2})$ bound from Lemma 11 can be applied (replacing n with $n/(3d \log_q n)^{1/d}$), making minor modifications to account for the fact that the alphabet here is slightly bigger, which can only make the problem harder. $\square$

This bound is tight up to poly-logarithmic factors, as by Lemma 6 a random pattern and text can be made injective at the cost of at most an $O(\log n)$ multiplicative factor, after which Theorem 12 can be applied.

5 Quantum Algorithm for Shift Finding in d Dimensions

In this section we describe a quantum algorithm for identifying hidden shifts in d dimensions, eventually proving the following theorem:

Theorem 4 (restated). *Let $d = O(1)$ and let X be an arbitrary finite set. Let $f : \mathbb{Z}_{2^n}^d \rightarrow X$ and $g : \mathbb{Z}_{2^n}^d \rightarrow X$ be injective functions such that $\Pr_x[g(x) \neq f(x+s)] = O(n^{-2} 2^{-\sqrt{(2 \log_2 3)dn}})$ for some $s \in \mathbb{Z}_{2^n}^d$. Then there is a quantum algorithm which outputs s with bounded error using $O(n 2^{\sqrt{(2 \log_2 3)dn}}) = O(n 2^{1.781 \dots \sqrt{dn}})$ queries to each of these functions. The runtime is the same up to a poly(n) factor.*

We first consider the case where g exactly matches f at some shift s . The algorithm to determine s can be seen as a hybrid of two algorithms of Kuperberg [21], so we begin by discussing the intuition behind Kuperberg’s original algorithm for the case $d = 1$ (see [21, 28] for more). It is based on producing a number of states

$$|\psi_r\rangle := \frac{1}{\sqrt{2}} (|0\rangle + \omega^{rs} |1\rangle)$$

for uniformly random $r \in \mathbb{Z}_{2^n}$, where we define $\omega := e^{\pi i / 2^{n-1}}$. We describe later on how this can be done. The algorithm uses a combination operation which takes as input two states $|\psi_r\rangle, |\psi_t\rangle$. This operation returns $|\psi_{r-t}\rangle$ with probability $1/2$ (we call this “success”), and $|\psi_{r+t}\rangle$ with probability $1/2$ (we call this “failure”). The intention is to produce the state $|\psi_{2^{n-1}}\rangle = \frac{1}{\sqrt{2}} (|0\rangle + (-1)^{s_n} |1\rangle)$, where s_n is the lowest-order bit of s . Given this state, the bit s_n can be determined by applying a Hadamard gate and measuring; using this as a subroutine turns out to be sufficient to identify the hidden shift s in its entirety.

The idea of Kuperberg [21] which will enable us to produce such a state $|\psi_{2^{n-1}}\rangle$ quite efficiently is as follows. Divide the n bits into $O(\sqrt{n})$ blocks of consecutive bits,

starting with the lowest-order bits. Each block is of length $O(\sqrt{n})$, aside from the last block, which only contains the highest-order bit. The algorithm is split into a number of stages. The i 'th stage of the algorithm is given as input a pool of states $|\psi_r\rangle$ such that all the bits of r in the first i blocks are zero. Then each state $|\psi_r\rangle$ is paired up with another state $|\psi_{r'}\rangle$ such that $r = r'$ on the bits in block $i + 1$, if such a state exists. The combination operation is applied to these pairs to produce some new states $|\psi_{r''}\rangle$ such that all the bits in the $(i + 1)$ 'st block of r'' are also 0. We repeat this process until we have zeroed all the bits, except the highest-order bit.

The probability that two random bit-strings agree in their lowest $O(\sqrt{n})$ bits is $2^{-O(\sqrt{n})}$. Thus it requires about $2^{O(\sqrt{n})}$ states to obtain “many” pairs whose lowest $O(\sqrt{n})$ bits are equal. Roughly a $1/2$ fraction of these pairs will successfully combine to give roughly a $1/4$ fraction of bit-strings which have their lowest-order bits zero, and can be input to the next stage. After applying the combination operation, the higher-order bits are still uniformly distributed, so we can repeat this argument. The net result is that we need to start with $2^{O(\sqrt{n})}$ states in total to have a good chance of eventually producing many states $|\psi_{2^{n-1}}\rangle$.

A similar idea can be used for the case $d > 1$. Here the hidden shift $s \in \mathbb{Z}_{2^n}^d$ is thought of as a d -tuple $(s^{(1)}, \dots, s^{(d)})$ of n -bit strings, and we redefine

$$|\psi_r\rangle := \frac{1}{\sqrt{2}} (|0\rangle + \omega^{r \cdot s} |1\rangle),$$

where $r = (r^{(1)}, \dots, r^{(d)})$ is a d -tuple of n -bit strings, and $r \cdot s = \sum_{i=1}^d r^{(i)} s^{(i)}$. We seek to learn the lowest-order bit $s_n^{(i)}$ of each bit-string. We achieve this by producing states $|\psi_r\rangle$ such that every bit of $r^{(i)}$ is zero for all i , except the highest-order bits $r_1^{(i)}$, which are uniformly random. Thus $|\psi_r\rangle = \frac{1}{\sqrt{2}} (|0\rangle + (-1)^{\sum_i r_1^{(i)} s_n^{(i)}} |1\rangle)$. If we measure in the Hadamard basis, we learn the sum modulo 2 of a random subset of the $s_n^{(i)}$ values; repeating this $O(d)$ times is sufficient to learn all the d bits $s_n^{(1)}, \dots, s_n^{(d)}$. In order to produce states $|\psi_r\rangle$ of this form, the same process as sketched above can be used to zero corresponding blocks of bits in every element of the d -tuple at once. It turns out to be more efficient to reduce the block size to $O(\sqrt{n/d})$; with this block size we end up with needing an initial pool of $2^{O(\sqrt{dn})}$ states.

The algorithm we give here achieves an improved complexity over Kuperberg's algorithm by noticing that the states $|\psi_{r+t}\rangle$ resulting from a failed combination operation between $|\psi_r\rangle$, $|\psi_t\rangle$ can be reused. If the pairs (r, t) and (r', t') all had the same low-order bits, so do the pair $(r + t, r' + t')$; and the high-order bits of $r + t$ and $r' + t'$ are still uniformly distributed. Assuming that we have $N \gg 2^{O(\sqrt{n})}$ states input to a given stage, we expect roughly $N/4 + N/16 + \dots = N/3$ states to be put through to the next stage, an improvement over the previous $N/4$. An additional improvement is found by modifying the block size depending on the algorithm's progress. In early stages, the algorithm has access to a very large pool of states, so should try to zero many bits at once. Later on, there are fewer states available, so fewer bits are zeroed.

Other algorithms Kuperberg describes a second algorithm which is based on greedily choosing states $|\psi_r\rangle$, $|\psi_t\rangle$ to combine, in order to maximise the number of

zero bits obtained [21]. The running time of his algorithm is essentially the same as the algorithm described here. However, the analysis of the algorithm given here seems (to the author) somewhat easier; and in particular, here we give a full proof that the algorithm succeeds with high probability, which is omitted in [21]. Kuperberg also outlines a general algorithm which works for any abelian group, rather than just the case $\mathbb{Z}_{2^n}^d$ we consider here. However, the complexity of this algorithm is not calculated precisely, only being given in the form $2^{O(\sqrt{n})}$. A subsequent algorithm of Regev [28] achieves improved space complexity over the algorithms of [21], but at the expense of increased time complexity. Another interesting algorithm for this problem, which achieves improved quantum (but not classical) space complexity, was recently given by Kuperberg [22]. This algorithm is believed to have a somewhat faster runtime compared with the algorithm given here, but only a heuristic argument for this is currently known [22].

5.1 The Algorithm

We now describe a quantum algorithm for solving the d -dimensional hidden shift problem. The algorithm is defined in terms of integers N and S , and a list of integers $b_1, \dots, b_S$ such that $\sum_{i=1}^S b_i = n - 1$. N is the number of states used, S is the number of stages of the algorithm, and b_i is the number of bits which are zeroed during stage i . At each stage i the algorithm operates on a list $\mathcal{L}_i$ of states, which are partitioned into bins.

The algorithm proceeds as follows:

1. Create a list $\mathcal{L}_1$ of N states $|\psi_r\rangle$, for random $r \in \mathbb{Z}_{2^n}^d$.
2. Repeat the following operations for $i = 1, \dots, S$:
 - (a) Sort each state $|\psi_r\rangle \in \mathcal{L}_i$ into one of 2^{db_i} bins according to the values of the bits of r at indices $n + 1 - \sum_{j=1}^i b_j, \dots, n - \sum_{j=1}^{i-1} b_j$.
 - (b) Repeatedly perform the following steps, until the total number of states in all the bins is at most n^2 :
 - i Divide the states in each bin into pairs, discarding any left-over states.
 - ii Apply the combination operation to each pair.
 - iii For each successful operation, add the resulting state to $\mathcal{L}_{i+1}$. For each failure, leave the resulting state in the same bin as before.
3. The result is a list $\mathcal{L}_{S+1}$ of states of the form $|\psi_r\rangle$, where r is uniformly distributed in $\{0, 2^{n-1}\}^d$.

Observe that the time complexity of the algorithm is $O(SN \log N)$, assuming that each element of the initial list can be created in time $O(1)$ and the combination operation takes time $O(1)$. This is because the time complexity of each stage is dominated by the sorting operation in step 2a, which can be carried out in time $O(N \log N)$. We have the following claim:

Claim 14 *For arbitrary $k = O(1)$, there exist $N = O(n2^{\sqrt{(2 \log_2 3)dn}})$, $S = O(\sqrt{n})$ and a choice of integers b_i such that the final list $\mathcal{L}_{S+1}$ satisfies $|\mathcal{L}_{S+1}| \geq k$ with probability at least $2/3$.*

We defer the proof of Claim 14 to “Appendix”. Assuming the correctness of this claim, we fill in the rest of the details about how the above algorithm can be used to solve the hidden shift problem, first showing how the states $|\psi_r\rangle$ can be produced. We begin by creating a number of quantum states of the form

$$\frac{1}{\sqrt{2^{dn+1}}} \sum_{x \in \mathbb{Z}_{2^n}^d} |x\rangle (|0\rangle |f(x)\rangle + |1\rangle |g(x)\rangle).$$

Such a state can be produced using one query to each of f and g . The last register is now measured and the remaining two registers are swapped (for notational clarity). Assuming the outcome $z \in X$ was received, the residual state will be

$$\frac{1}{\sqrt{2}} (|0\rangle |f^{-1}(z)\rangle + |1\rangle |g^{-1}(z)\rangle) = \frac{1}{\sqrt{2}} (|0\rangle |a\rangle + |1\rangle |a-s\rangle),$$

for some element $a = f^{-1}(z)$, using injectivity of f and g and the fact that $g(x) = f(x+s)$. The next step is to perform the inverse quantum Fourier transform over $\mathbb{Z}_{2^n}^d$ on the second register. This is equivalent to performing the tensor product of d inverse QFTs over $\mathbb{Z}_{2^n}$. The result is the state

$$\begin{aligned} & \frac{1}{\sqrt{2^{dn+1}}} \left(\sum_{x \in \mathbb{Z}_{2^n}^d} \omega^{-x \cdot a} |0\rangle |x\rangle + \sum_{x \in \mathbb{Z}_{2^n}^d} \omega^{-x \cdot (a-s)} |1\rangle |x\rangle \right) \\ &= \frac{1}{\sqrt{2^{dn+1}}} \sum_{x \in \mathbb{Z}_{2^n}^d} (|0\rangle + \omega^{x \cdot s} |1\rangle) \omega^{-x \cdot a} |x\rangle, \end{aligned}$$

where as before $\omega := e^{\pi i / 2^{n-1}}$ and $x \cdot a = \sum_{i=1}^d x^{(i)} a^{(i)}$, with multiplication taken over $\mathbb{Z}_{2^n}$. We now measure the second register and are left with the residual state

$$|\psi_r\rangle := \frac{1}{\sqrt{2}} (|0\rangle + \omega^{r \cdot s} |1\rangle)$$

for some $r \in \mathbb{Z}_{2^n}^d$. Note that r is uniformly random and we know what it is. We now describe how the combination operation works. Recall that, given two states $|\psi_r\rangle, |\psi_t\rangle$, this operation should produce $|\psi_{r-t}\rangle$ with probability $1/2$, and otherwise produce $|\psi_{r+t}\rangle$, where addition and subtraction are over $\mathbb{Z}_{2^n}^d$. This is simply achieved by measuring the parity of the two qubits; with probability $1/2$ we get “even” and the state collapses to $\frac{1}{\sqrt{2}}(|00\rangle + \omega^{(r+t) \cdot s} |11\rangle)$, and with probability $1/2$ we get “odd” and the state collapses to $\frac{1}{\sqrt{2}}(\omega^{r \cdot s} |10\rangle + \omega^{t \cdot s} |01\rangle)$. By relabelling basis states, and ignoring an overall phase factor, these are equivalent to $|\psi_{r+t}\rangle$ and $|\psi_{r-t}\rangle$ respectively.

Using the above algorithm, for arbitrary $k = O(1)$ we can produce a list of k states of the form $|\psi_r\rangle$, where r is uniformly distributed in $\{0, 2^{n-1}\}^d$. Defining $\beta \in \{0, 1\}^d$ by $\beta_i = r_n^{(i)}$, we have $|\psi_r\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^{\beta \cdot s'} |1\rangle)$, where $s' \in \{0, 1\}^d$ is the vector

of lowest-order bits of s and $\beta \cdot s' = \sum_{i=1}^d \beta_i s'_i$. Therefore, if we apply a Hadamard gate and measure, we learn the value of $\beta \cdot s'$ with certainty, for a random $\beta \in \{0, 1\}^d$ (which we know). It suffices to do this $O(d)$ times to learn s' completely with high probability. Picking $k = O(d)$, one run of the algorithm is enough to learn s' with bounded failure probability.

Once the lowest-order bit of each component of s has been learned, the remaining bits can be learned using the following idea. Let $\beta \in \{0, 1\}^d$ be the lowest-order bits, and let $a \in \{0, 1\}^d$ be arbitrary. Define $f'(x) = f(2x + a)$ for $x \in \mathbb{Z}_{2^{n-1}}^d$, and $g'(x) = g(2x + a - \beta)$. Also let $s' \in \mathbb{Z}_{2^n}^d$ be the d -tuple obtained by setting $(s')^{(i)} = \lfloor s^{(i)} / 2 \rfloor$. Then we have

$$\begin{aligned} g'(x) &= g(2x + a - \beta) = f(2x + a - \beta + 2s' + \beta) \\ &= f(2(x + s') + a) = f'(x + s'). \end{aligned}$$

We can therefore apply the above algorithm to f' and g' to find the lowest-order bits of s' , repeating this procedure to learn s completely.

5.2 The Approximate Case

We now show that the above algorithm still works when g only approximately matches f . The algorithm for obtaining the lowest-order bits of the shift s is based on producing $O(n2^{\sqrt{(2 \log_2 3)dn}})$ quantum states of the form

$$|\phi_r\rangle := \frac{1}{\sqrt{2}} (|0\rangle|f(r)\rangle + |1\rangle|g(r)\rangle)$$

for uniformly random $r \in \mathbb{Z}_{2^n}^d$, and applying some processing to these states. One can thus see the algorithm as operating on $O(n2^{\sqrt{(2 \log_2 3)dn}})$ copies of a mixed state

$$\rho := \frac{1}{2^{nd}} \sum_{r \in \mathbb{Z}_{2^n}^d} |r\rangle\langle r| \otimes |\phi_r\rangle\langle \phi_r|.$$

If f and g are modified to take different values at up to $\epsilon 2^{nd}$ positions each, and we let $\tilde{\rho}$ denote the corresponding mixed state, we have $\|\tilde{\rho} - \rho\|_1 \leq 2\epsilon$. This implies that, if f and g are modified by changing their values at up to an arbitrary $O(\delta n^{-1} 2^{\sqrt{(2 \log_2 3)dn}})$ fraction of positions, for arbitrary $0 \leq \delta < 1$, with probability at least $1 - \delta$ the algorithm does not notice the difference between having copies of ρ and copies of $\tilde{\rho}$ and will still output the lowest-order bits of the (necessarily unique) shift s maximising $|\{x : g(x) = f(x + s)\}|$.

In each subsequent iteration, the algorithm solves the same problem for functions whose domain is half the size of the previous iteration, based on the previous functions evaluated at positions determined by an arbitrary offset $a \in \{0, 1\}^d$. Consider the first

such choice and let $f_a, g_a : \mathbb{Z}_{2^{n-1}}^d \rightarrow X$ denote the new functions. If we pick a uniformly at random, we have

$$\mathbb{E}_a[\Pr_x[f_a(x) \neq g_a(x)]] = \epsilon.$$

Therefore, by Markov's inequality $\Pr_a[\Pr_x[f_a(x) \neq g_a(x)] \geq \alpha\epsilon] \leq 1/\alpha$ for any $\alpha \geq 1$. The same argument holds for all subsequent iterations of the algorithm too. Using a union bound over the $O(n)$ iterations, taking $\alpha = O(n)$ and $\epsilon = O(n^{-2}2^{-\sqrt{(2\log_2 3)^{dn}}})$ suffices to ensure that the algorithm succeeds with probability $\Omega(1)$.

6 Outlook

We have described a quantum algorithm which achieves a super-polynomial separation from classical computation for the basic problem of pattern matching on average-case inputs. There are some undesirable aspects of our algorithm. First, the super-polynomial speedup is only achieved if we accept that for certain inputs, the algorithm might fail with high probability. This cannot be avoided: even checking whether a claimed match is really a match must take time $\Omega(m^{d/2})$ in the worst case. However, observe that the algorithm of Theorem 2 does have the property that, if it outputs an offset at which the pattern is claimed to match the text, we can be confident that this only differs from a real match at an $O(\gamma)$ fraction of positions.

Second, the $2^{O(d^{3/2}\sqrt{\log m})}$ component of the runtime, while $o(m^\epsilon)$ for any $\epsilon > 0$, is still undesirably high. Substantially improving this term (to be logarithmic in m , for example) would presumably require finding an efficient quantum algorithm for the dihedral hidden subgroup problem, which has been a major open problem for over a decade. However, as the algorithm for finding hidden shifts is used as a black box, any improvements to this would imply an improved pattern-matching algorithm.

Finally, an interesting open question is whether efficient quantum algorithms can be found for approximate pattern matching. In the classical literature, Chang and Lawler describe an approximate pattern matching algorithm running in time $O((n/m) \log m)$ on random inputs, if one ignores the time to preprocess the pattern [7]. Another example is the classical work of Andoni et al. [2] on the noisy hidden shift problem. The quantum algorithm described in Sect. 5 can be used to solve the noisy hidden shift problem for random inputs as long as the noise rate is very low ($2^{-O(\sqrt{\log n})}$ for a problem of size n). Solving the hidden shift problem with a constant noise rate more efficiently than is possible classically seems likely to require new ideas.

Acknowledgements I would like to thank Raphaël Clifford, Markus Jalsenius and Ben Sach for helpful discussions on the topic of this paper. I would also like to thank one pseudonymous and two anonymous referees for a number of detailed comments and suggestions which have significantly improved the paper. This work was supported by an EPSRC Early Career Fellowship (EP/L021005/1).

Appendix: Proof of Claim 14

In this appendix, we analyse the algorithm of Sect. 5 for identifying hidden shifts. Let $L_i = |\mathcal{L}_i|$ denote the number of states in the list at the start of the i 'th stage. Our first task is to find a lower bound on L_{i+1} in terms of L_i which holds with high probability. For ease of analysis, we consider a different, “relaxed” process where any state is allowed to be combined with any other state, rather than being restricted to the same bin; and where we assume there are always an even number of states. We then relate the relaxed process to the real process followed by the algorithm by bounding the number of states that the real process would need to discard, given its restriction to only combining states in the same bin.

Let $L_i^{(j)}$ denote the number of states we would have in the list $\mathcal{L}_i$ after j steps of the i 'th stage, based on following the relaxed process; note that $L_i^{(0)} = L_i$. Also let $S_i^{(j)}$ denote the number of successful pairings in the j 'th step of the i 'th stage (again allowing any pair of states to be combined). Finally let t_i denote the number of steps taken in the i 'th stage (i.e. until the total number of states is at most n^2). Then

$$L_i^{(j)} = \frac{L_i^{(j-1)}}{2} - S_i^{(j)}, \quad L_{i+1} \geq \sum_{j=1}^{t_i} S_i^{(j)} - t_i 2^{db_i}, \quad (1)$$

where the second inequality allows for the fact that at each step we may need to discard at most 2^{db_i} states in the real process, as compared with the relaxed process. The probability that each combination operation applied to a pair succeeds is independent and equal to $1/2$. Using a Chernoff bound, we have

$$\Pr[|S_i^{(j)} - L_i^{(j-1)}/4| \geq \ln n \sqrt{L_i^{(j-1)}/4}] \leq 2e^{-(\ln n)^2/2} = 2n^{-(\ln n)/2}. \quad (2)$$

As there will be $\text{poly}(n)$ steps in total throughout the algorithm, this quantity is small enough that we can take a union bound over all steps and assume that this event never happens. Making this assumption, we have

$$L_i^{(j+1)} \geq \frac{L_i^{(j)}}{2} - \frac{L_i^{(j)}}{4} - \frac{\ln n}{2} \sqrt{L_i^{(j)}} = \frac{L_i^{(j)}}{4} \left(1 - \frac{2 \ln n}{\sqrt{L_i^{(j)}}} \right). \quad (3)$$

As we have $L_i^{(j)} \geq n^2$ for all steps j , then

$$L_i^{(j+1)} \geq \frac{L_i^{(j)}}{4} \left(1 - \frac{2 \ln n}{n} \right),$$

implying

$$L_i^{(j)} \geq L_i \left(\frac{1 - (2 \ln n)/n}{4} \right)^j. \quad (4)$$

By (1), (2) and (3),

$$\begin{aligned} L_{i+1} &\geq \sum_{j=1}^{t_i} S_i^{(j)} - t_i 2^{db_i} \geq \sum_{j=1}^{t_i} \frac{L_i^{(j-1)}}{4} \left(1 - \frac{2 \ln n}{\sqrt{L_i^{(j-1)}}} \right) \\ &\quad - t_i 2^{db_i} \geq \frac{1}{4} \sum_{j=1}^{t_i} L_i^{(j-1)} \left(1 - \frac{2 \ln n}{n} \right) - t_i 2^{db_i} \end{aligned}$$

and so by (4), writing $\xi = (2 \ln n)/n$,

$$\begin{aligned} L_{i+1} &\geq \frac{L_i}{4} (1 - \xi) \sum_{j=0}^{t_i-1} \left(\frac{1 - \xi}{4} \right)^j - t_i 2^{db_i} = \frac{L_i}{4} (1 - \xi) \frac{1 - ((1 - \xi)/4)^{t_i}}{1 - (1 - \xi)/4} - t_i 2^{db_i} \\ &= L_i \frac{(1 - \xi)(1 - ((1 - \xi)/4)^{t_i})}{3 + \xi} - t_i 2^{db_i}. \end{aligned}$$

A rough lower bound that follows from (3) for large enough n is that $L_i^{(j+1)} \geq L_i^{(j)}/8$; and as $L_i^{(j+1)} \leq L_i^{(j)}/2$ always, we have $1/8 \leq L_i^{(j+1)}/L_i^{(j)} \leq 1/2$. Using $L_i^{(0)} = L_i$ and $L_i^{(t_i)} \leq n^2$ we obtain $\log_2(L_i/n^2) \leq t_i \leq 3 \log_2(L_i/n^2)$. So

$$((1 - \xi)/4)^{t_i} \leq 4^{-t_i} \leq n^4/L_i^2.$$

Assuming that $L_i \geq n^3$ for all $1 \leq i \leq S$, we have

$$\frac{(1 - \xi)(1 - ((1 - \xi)/4)^{t_i})}{3 + \xi} \geq \frac{(1 - (2 \ln n)/n)(1 - 1/n^2)}{3 + (2 \ln n)/n} = \frac{1 - O((\log n)/n)}{3}.$$

Further assume that $L_i = 2^{O(\sqrt{n})}$ for all i , implying $t_i = O(\sqrt{n})$. Then

$$L_{i+1} \geq \frac{(1 - O((\log n)/n))L_i}{3} - O(\sqrt{n})2^{db_i}.$$

We now need to determine how large L_1 needs to be such that L_{S+1} is still quite large. Working backwards, we can take

$$L_{i-1} \leq 3(1 + O((\log n)/n))L_i + O(\sqrt{n})2^{db_{i-1}}.$$

Thus

$$\begin{aligned} L_1 &\leq (3(1 + O((\log n)/n)))^S L_{S+1} + (3(1 + O((\log n)/n)))^{S-1} O(\sqrt{n})2^{db_S} \\ &\quad + \cdots + O(\sqrt{n})2^{db_1}. \end{aligned}$$

Assuming that $L_{S+1} = O(1)$, and $S = O(\sqrt{n})$, we have

$$L_1 = O\left(\sqrt{n} \sum_{i=1}^S 3^i 2^{db_i}\right)$$

We now need to pick values S , b_i for the number of stages and the number of bits zeroed at each stage, such that $\sum_{i=1}^S b_i = n - 1$, to minimise $\sum_{i=1}^S 3^i 2^{db_i}$. We choose these values to make all the above terms equal to $2^{c\sqrt{n}}$ for some fixed c , i.e. $b_i = (c\sqrt{n} - (\log_2 3)i)/d$ (for simplicity ignoring the fact that b_i has to be rounded to an integer). Relaxing to the constraint $\sum_i b_i = n$ for simplicity, we obtain $Sc\sqrt{n} - (\log_2 3)S(S+1)/2 = dn$. Hence $c = d\sqrt{n}/S + (\log_2 3)(S+1)/(2\sqrt{n})$. Minimising this over S , we get that the minimum is found at $S = \sqrt{2\log_3 2\sqrt{dn}}$, giving

$$c = \frac{\sqrt{d}}{\sqrt{2\log_3 2}} + \frac{\log_2 3(\sqrt{2\log_3 2\sqrt{dn}} + 1)}{2\sqrt{n}} = \sqrt{(2\log_2 3)d} + O(1/\sqrt{n}).$$

Thus

$$L_1 = O(\sqrt{n} S 2^{(\sqrt{2(\log_2 3)d} + O(1/\sqrt{n}))\sqrt{n}}) = O(n 2^{\sqrt{(2\log_2 3)dn}}) = O(n 2^{1.781\dots\sqrt{dn}})$$

as claimed.

References

1. Ambainis, A.: Quantum lower bounds by quantum arguments. *J. Comput. Syst. Sci.* **64**, 750–767. [quant-ph/0002066](#) (2002)
2. Andoni, A., Hassanieh, H., Indyk, P., Katabi, D.: Shift finding in sub-linear time. In: *Proceedings of 24th ACM-SIAM Symposium on Discrete Algorithms*, pp. 457–465, (2013)
3. Batu, T., Ergün, F., Kilian, J., Magen, A., Raskhodnikova, S., Rubinfeld, R., Sami, R.: A sublinear algorithm for weakly approximating edit distance. In: *Proceedings of 35th Annual ACM Symposium on Theory of Computing*, pp. 316–324, (2003)
4. Bennett, C., Bernstein, E., Brassard, G., Vazirani, U.: Strengths and weaknesses of quantum computing. *SIAM J. Comput.* **26**(5), 1510–1523. [quant-ph/9701001](#) (1997)
5. Boyer, R., Moore, S.: A fast string searching algorithm. *Commun. ACM* **20**(10), 762–772 (1977)
6. Brassard, G., Høyer, P., Mosca, M., Tapp, A.: Quantum Amplitude Amplification and Estimation. *Quantum Computation and Quantum Information: A Millennium Volume*, pp. 53–74. [quant-ph/0005055](#) (2002)
7. Chang, W., Lawler, E.: Sublinear approximate string matching and biological applications. *Algorithmica* **12**(4–5), 327–344 (1994)
8. Childs, A., van Dam, W.: Quantum algorithm for a generalized hidden shift problem. In: *Proceedings of 18th ACM-SIAM Symposium on Discrete Algorithms*, pp. 1225–1232. [quant-ph/0507190](#) (2007)
9. Childs, A., Kothari, R., Ozols, M., Roetteler, M.: Easy and hard functions for the boolean hidden shift problem. In: *Proceedings of 8th Conference on the Theory of Quantum Computation, Communication, and Cryptography (TQC'13)*, pp. 50–79. [arXiv:1304.4642](#) (2013)
10. Crochemore, M., Hancart, C., Lecroq, T.: *Algorithms on Strings*. Cambridge University Press, Cambridge (2007)
11. Curtis, D., Meyer, D.: Towards quantum template matching. In: *Proceedings of SPIE 5161, Quantum Communications and Quantum Imaging*, pp. 134–141, (2004)

12. Ettinger, M., Høyer, P., Knill, E.: The quantum query complexity of the hidden subgroup problem is polynomial. *Inf. Process. Lett.* **91**, 43–48 (2004). [quant-ph/0401083](#)
13. Friedl, K., Ivanyos, G., Magniez, F., Santha, M., Sen, P.: Hidden translation and orbit coset in quantum computing. In: *Proceedings of 35th Annual ACM Symposium on Theory of Computing*, pp. 1–9. [quant-ph/0211091](#) (2003)
14. Gavinsky, D., Roetteler, M., Roland, J.: Quantum algorithm for the Boolean hidden shift problem. In: *Proceedings of 17th International Computing & Combinatorics Conference (COCOON'11)*, pp. 158–167. [arXiv:1103.3017](#) (2011)
15. Gharibi, M.: Reduction from non-injective hidden shift problem to injective hidden shift problem. *Quantum Inf. Comput.* **13**(3&4), 221–230 (2013). [arXiv:1207.4537](#)
16. Giovannetti, V., Lloyd, S., Maccone, L.: Quantum random access memory. *Phys. Rev. Lett.* **100**, 160501 (2008). [arXiv:0708.1879](#)
17. Grover, L.: Quantum mechanics helps in searching for a needle in a haystack. *Phys. Rev. Lett.* **79**(2), 325–328 (1997). [quant-ph/9706033](#)
18. Høyer, P., Mosca, M., de Wolf, R.: Quantum search on bounded-error inputs. In: *Proceedings of 30th International Conference on Automata, Languages and Programming (ICALP'03)*, pp. 291–299. [quant-ph/0304052](#) (2003)
19. Kärkkäinen, J., Ukkonen, E.: Two- and higher-dimensional pattern matching in optimal expected time. *SIAM J. Comput.* **29**(2), 571–589 (1999)
20. Knuth, D., Morris Jr, J., Pratt, V.: Fast pattern matching in strings. *SIAM J. Comput.* **6**(2), 323–350 (1977)
21. Kuperberg, G.: A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM J. Comput.* **35**(1), 170–188 (2005). [quant-ph/0302112](#)
22. Kuperberg, G.: Another subexponential-time quantum algorithm for the dihedral hidden subgroup problem. In: *Proceedings of 8th Conference on the Theory of Quantum Computation, Communication, and Cryptography (TQC'13)*, pp. 20–34. [arXiv:1112.3333](#) (2013)
23. Montanaro, A., de Wolf, R.: Quantum property testing. [arXiv:1310.2035](#) (2013)
24. Moore, C., Rockmore, D., Russell, A., Schulman, L.: The power of strong fourier sampling: quantum algorithms for affine groups and hidden shifts. *SIAM J. Comput.* **37**(3), 938–958 (2005). [quant-ph/0503095](#)
25. Navarro, G.: A guided tour to approximate string matching. *ACM Comput. Surv.* **33**(1), 31–88 (2001)
26. Ozols, M., Roetteler, M., Roland, J.: Quantum rejection sampling. *ACM Trans. Comput. Theory (TOCT)* **5**(3), 11:1–11:33 (2013). [arXiv:1103.2774](#)
27. Ramesh, H., Vinay, V.: String matching in $\tilde{O}(\sqrt{n} + \sqrt{m})$ quantum time. *J. Discrete Algorithms* **1**, 103–110 (2003). [quant-ph/0011049](#)
28. Regev, O.: A subexponential time algorithm for the dihedral hidden subgroup problem with polynomial space. [quant-ph/0406151](#) (2004)
29. Rötteler, M.: Quantum algorithms to solve the hidden shift problem for quadratics and for functions of large Gowers norm. In: *Proceedings of 39th International Symposium on Mathematical Foundations of Computer Science (MFCS'09)*, LNCS vol. 5734, pp. 663–674. [arXiv:0911.4724](#) (2009)
30. Rötteler, M.: Quantum algorithms for highly non-linear Boolean functions. In: *Proceedings of 21st ACM-SIAM Symposium on Discrete Algorithms*, pp. 448–457. [arXiv:0811.3208](#) (2010)
31. Twamley, J.: A hidden shift quantum algorithm. *J. Phys. A Math. Gen.* **33**, 8973 (2000)
32. van Dam, W., Hallgren, S., Ip, L.: Quantum algorithms for some hidden shift problems. *SIAM J. Comput.* **36**, 763–778 (2006). [quant-ph/0211140](#)
33. Vishkin, U.: Deterministic sampling—a new technique for fast pattern matching. *SIAM J. Comput.* **20**(1), 22–40 (1991)
34. Yao, A.: The complexity of pattern matching for a random string. *SIAM J. Comput.* **8**(3), 368–387 (1979)